

Fraud Detection AI Agent

Project overview: the data, what we are building, and where it stands

Domain	Insurance fraud (India): policies, claims, premium payments, agents
Data	4 labelled CSV files, 1,000 records each (synthetic)
Platform	Google Cloud: BigQuery (Mumbai, asia-south1) + Vertex AI
Agent framework	Google Agent Development Kit (ADK) 2.9, Gemini 3.8 Flash
GCP project	project-8e0caa5b-233c-4e40-823
Status	Phase 1 (data + rules) done, Phase 2 (claims model) done, Phase 3 (agents) in progress
Date	18 September 2026

Contents

- 1. The big picture
- 2. The four CSV files in detail
- 3. What the analysis found
- 4. What we are building: architecture
- 5. Technology choices
- 6. Phase 1: loading the data into BigQuery
- 7. Phase 1: the rules engine
- 8. Phase 2: the claims model
- 9. Phase 3: the claims agent
- 10. The full multi-agent design (target)
- 11. Status, open issues and next steps
- 12. How to run everything
- 13. Glossary

1. The big picture

Insurance fraud is when someone gets money or cover from an insurer that they are not entitled to. It happens at every stage of an insurance relationship: when a policy is sold, when the premium is paid, and when a claim is made. It can be committed by customers, by the agents who sell policies, or by people pretending to be agents.

This project builds a system that **finds suspected fraud automatically, investigates it, and explains the evidence to a human investigator**, who makes the final decision. The key idea is to use each technique for what it is good at:

- **Rules** catch the obvious, certain cases (for example, a receipt number that was already used). They are fast, cheap, exact and easy to explain.
- **A machine-learning (ML) model** scores every record for how suspicious it looks, catching combinations of weaker signals that no single rule captures.
- **AI agents** (large language models that can call tools) investigate the flagged cases: they look up the policy, the customer's history, the agent's record and related claims across all the data, and write up a case with evidence.
- **A human investigator** reviews the case and decides. Their decisions become new training data.

Why not just ask an AI to read every row?

It would be slow and expensive, and language models are not reliable at spotting patterns in thousands of rows. Rules and ML do the screening; the AI does what it is good at: gathering context, reasoning about it, and explaining it in plain language.

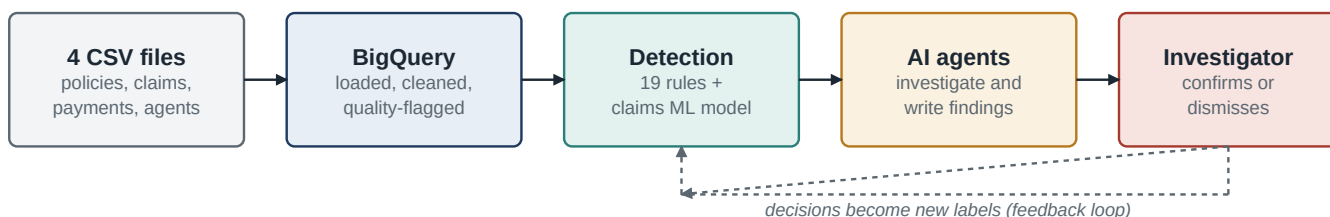


Figure 1. The end-to-end flow. Data is loaded and cleaned in BigQuery, screened by rules and ML, investigated by AI agents, and decided by a person. Decisions feed back into the models.

2. The four CSV files in detail

There are four files. Each has 1,000 rows, no missing values (except one expected case), and two **label** columns: `fraud_flag` (TRUE/FALSE, was this record fraud?) and `fraud_type` (which kind). Labels are the “answers” we train and test against. The data is **synthetic** (generated, not real) and set in the Indian market: amounts are in rupees, and it uses Indian states, cities and payment modes such as UPI and NEFT.

How the files connect

The four files share three identifiers, which lets us follow a person or an agent across all of them:

- `policy_id` (POL000001...): the insurance contract. **policies_free_insurance.csv** is the master list of policies.
- `customer_id` (CUST000001...): the policyholder.
- `agent_id` (AGT00001...): the salesperson who sold or handles the policy. 222 agents appear in all four files.

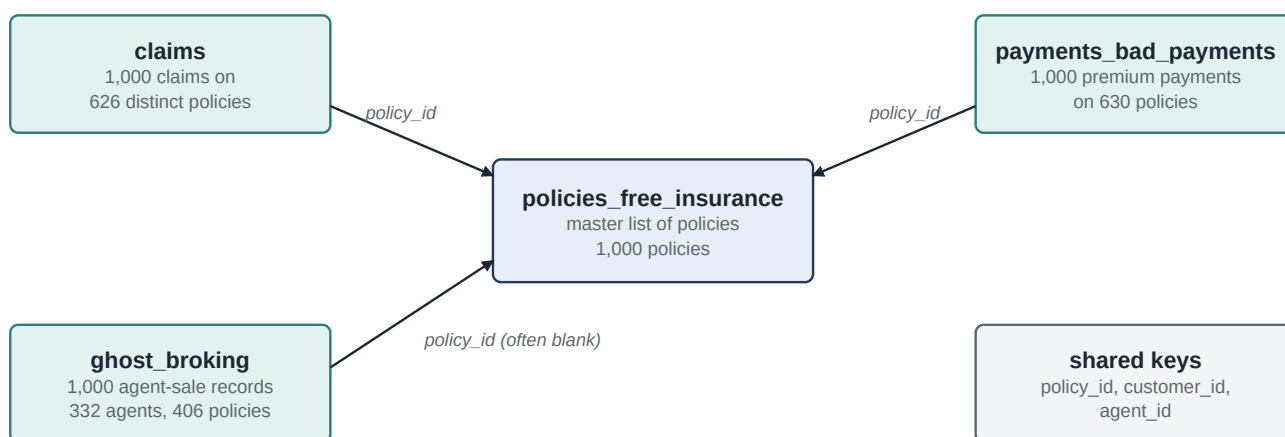


Figure 2. How the files link. Every claim and payment points to a policy in the policies file; 405 of 406 policy IDs in ghost_broking do too (438 ghost-broking rows have no policy at all, which is expected for fake sales). Customers and agents link across all four files.

2.1 policies_free_insurance.csv — “getting cover without really paying”

One row per insurance policy. The fraud here is about **obtaining insurance cover you have not properly paid for**, or policies that are not what they seem. 155 of 1,000 policies (15.5%) are fraud.

Column	Meaning
<code>policy_id</code> , <code>customer_id</code> , <code>agent_id</code>	The policy, who holds it, who sold it
<code>policy_type</code>	Auto (347), Health (254), Life (155), Property (138), Travel (106)
<code>issue_date</code>	When the policy started (DD-MM-YYYY in the file)
<code>premium_amount</code> / <code>sum_insured</code>	Price paid per period / maximum payout (rupees)
<code>policy_status</code>	Active, Cancelled, Lapsed, Matured
<code>channel</code> , <code>state</code>	How it was sold (Agent, Online, Broker, Bancassurance) and the Indian state
<code>free_look_period_days</code>	Always 15: the legal window in which a new policy can be cancelled for a refund
<code>free_look_cancelled</code>	Was it cancelled inside that window?
<code>backdated_flag</code>	Was the start date set earlier than the real sale date?
<code>premium_actually_collected</code>	Did the insurer ever actually receive the premium?

Fraud type	Count	What it means
Fake Policy – No Premium Collected	45	A policy exists on paper but no premium was ever collected
Backdated Policy Issuance	41	Start date moved back so an event that already happened is “covered”
Fronting / Straw Policyholder	39	The named holder is a front for someone else (e.g. someone who could not get cover)

Fraud type	Count	What it means
Free-Look Period Abuse	30	Policy bought and cancelled in the free-look window to exploit the refund or commission

2.2 claims.csv — fraudulent claims

One row per claim made on a policy. The fraud here is about **getting paid for a loss that did not happen, was staged, or was inflated**. 184 of 1,000 claims (18.4%) are fraud.

Column	Meaning
claim_id, policy_id, customer_id, agent_id	The claim and who/what it relates to
claim_type	Vehicle Damage/Accident (329), Hospitalization (272), Death (172), Fire/Theft/Damage (122), Trip Cancellation/Medical (105)
incident_date / claim_date	When the loss happened / when it was reported
days_policy_to_incident	Days from policy start to the incident (very small = suspicious)
claim_amount	Amount claimed (rupees)
approved_amount, claim_status	What the insurer paid and the claim's status. These are outcomes of an investigation and are never used as evidence (see section 3)
num_claimsFiled_by_customer	How many claims this customer has filed
police_reportFiled, witness_count	Supporting evidence for the incident
documents_altered_flag	Were the supporting documents found to be altered?

Fraud type	Count	What it means
Multiple/Duplicate Claims	52	The same loss claimed more than once
Staged Accident	39	An accident deliberately caused or faked
Fake Supporting Documents	34	Forged or edited bills, reports or certificates
Exaggerated Claim Amount	33	A real loss, but the amount inflated
Early Claim Fraud	26	Policy taken out just before a loss that was already known or planned

2.3 payments_bad_payments.csv — bad premium payments

One row per premium payment. The fraud here is about **money that never really reached the insurer**, or was taken back. 144 of 1,000 payments (14.4%) are fraud.

Column	Meaning
payment_id, policy_id, customer_id, agent_id	The payment and who/what it relates to
payment_date, payment_amount	When and how much (rupees)
payment_mode	UPI (294), Card (252), Cheque (219), Cash (130), NEFT (105)
payment_status	Success (921), Reversed (44), Bounced (35)
receipt_number, is_duplicate_receipt	Receipt issued, and whether that receipt number was already used
premium_remitted_to_insurer	Did the money collected actually reach the insurer?
remittance_delay_days	Days between collection and handing the money to the insurer

Fraud type	Count	What it means
Duplicate/Unauthorized Refund Claimed	44	A refund taken that was not owed (shows as a Reversed payment)
Fake Payment Receipt Submitted	36	A copied or forged receipt used as proof of payment
Bounced Cheque Used for Coverage	35	Cover obtained with a cheque that later bounced
Premium Diversion by Agent	29	The agent kept the customer's premium instead of passing it on

2.4 ghost_broking.csv — fake and unlicensed agents

“Ghost broking” is when someone sells insurance without the right to: a fake or cloned agent identity, an expired or suspended license, or no license at all. Customers often think they are insured when they are not. One row per sale by an agent. 285 of 1,000 records (28.5%) are fraud.

Column	Meaning
record_id, agent_id, agent_name, customer_id	The sale record and who was involved
policy_id	The policy sold. Blank in 438 rows (fake sales often have no real policy)
license_status	Valid (628), Fake/Cloned (103), Suspended (97), Never Issued (93), Expired (79)
license_expiry_date	When the agent's license expires or expired
premium_collected	Money the agent took from the customer (rupees)
premium_deposited_with_insurer	Did the agent pass the money on?
customer_aware_of_agent_status	Did the customer know the agent's real license status?
complaint_count, agent_office_city	Complaints against the agent on this record; the agent's city (15 cities)

Fraud type	Count	What it means
Fake/Cloned Agent Identity	81	Someone posing as a licensed agent using a stolen or invented identity
Premium Pocketing – No Remittance	80	Agent collected premium and kept it; the customer has no cover
Expired License Sales	70	Selling after the license expired
Unlicensed Selling	54	Selling with no license at all

3. What the analysis found

Before building anything, every file was profiled: fraud rates for every value of every column, consistency checks between files, and date checks. Four findings shaped the whole design.

Finding 1: most fraud types are “given away” by one column

Many fraud types match a single column exactly. Every row with that flag is fraud, and every fraud of that type has the flag:

Column value	File	Share that is fraud	Fraud type it reveals
backdated_flag = TRUE	policies	100% (41)	Backdated Policy Issuance
premium_actually_collected = FALSE	policies	100% (45)	Fake Policy – No Premium Collected
documents_altered_flag = TRUE	claims	100% (34)	Fake Supporting Documents
is_duplicate_receipt = TRUE	payments	100% (36)	Fake Payment Receipt Submitted
premium_remitted_to_insurer = FALSE	payments	100% (29)	Premium Diversion by Agent
payment_status = Bounced / Reversed	payments	100% (35 / 44)	Bounced Cheque / Refund fraud
premium_deposited_with_insurer = FALSE	ghost_broking	100% (80)	Premium Pocketing

That makes these types easy to catch with rules. It also means a model trained on these columns would score close to 100% while learning nothing useful: real data is never this clean.

Finding 2: some columns leak the answer (“label leakage”)

In the claims file, `claim_status` is 100% fraud when it is Rejected or Under Investigation, and 0% fraud when Approved. The status is the *result* of a fraud investigation, not evidence for one. The same is true of `approved_amount` (fraud claims were paid far less). Using these columns would be cheating: at the moment a new claim arrives, they do not exist yet. They are therefore **excluded from every rule, model and agent tool**, and code tests enforce this.

Finding 3: some fraud types have no signal in the data

Staged accidents, duplicate claims and fronting look statistically identical to honest records. Witness counts, police reports, report delays and repeat-claim counts are flat across fraud and non-fraud. For example, 51% of honest claims have another related claim within 180 days, almost the same as duplicate-claim fraud (62%), and no two claims share an amount or incident date. Real data would need extra signals (repair-shop or hospital IDs, vehicle numbers, document text) to catch these.

Finding 4: data-quality problems

Problem	Where	Count	How it is handled
Claim or incident dates in the future (up to May 2029)	claims	92 claims	Flagged (<code>dq_future_claim_date</code>), kept
“Valid” license whose expiry date has passed	ghost_broking	603 of 628	Flagged (<code>dq_valid_license_expired</code>)
Same agent with different license statuses	ghost_broking	50 agents	Flagged (<code>dq_agent_license_conflict</code>)
Blank <code>policy_id</code>	ghost_broking	438 rows	Converted to NULL (expected for fake sales)
<code>free_look_period_days</code> is always 15	policies	all	Ignored: carries no information

Important caveat

Because the data is synthetic, the pipeline and design will carry over to real data, but the specific accuracy numbers in this document will not. Real fraud is noisier.

4. What we are building: architecture

The system has four layers. Data flows downward; decisions flow back up.

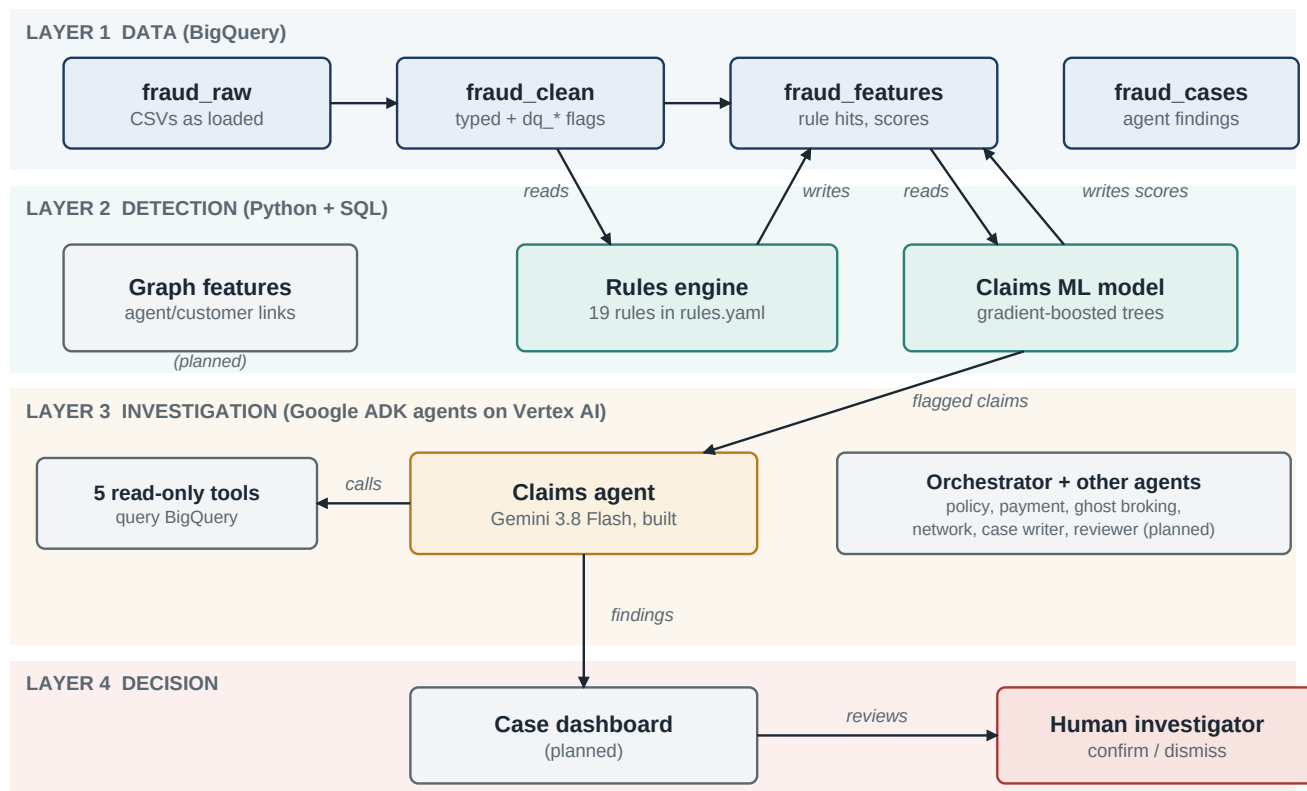


Figure 3. The four layers. Solid boxes are built and running; grey boxes are planned. Every layer reads from and writes to BigQuery, so any stage can be re-run independently.

Why this split works

Layer	Good at	Weak at
Rules	Certain cases; instant; free; every hit is explainable	Only catches what someone thought to write down
ML model	Combining many weak signals into one score	Cannot explain itself in words; needs labelled data
AI agents	Gathering context across files; reasoning; writing a readable case	Slow and costly per case; can over-call if not calibrated
Humans	Final judgement, accountability, catching new patterns	Expensive; cannot look at every record

5. Technology choices

Layer	Choice	Why
Language	Python 3.12, managed with uv	Standard for data and ML; Google Cloud libraries support it (3.14 is too new)
Data store	Google BigQuery, Mumbai (asia-south1)	Managed and shared by every service; access control and audit logs; data stays in India; free tier covers this size
Rules	YAML file + SQL generated by Python	Change rules without touching code; rules run inside BigQuery
ML	scikit-learn gradient boosting	Strong on tabular data; no extra system libraries (LightGBM needs one on macOS)
Agents	Google ADK 2.9 on Vertex AI	Uses the Google Cloud credits; tools are plain Python functions; structured output
Model	gemini-3.8-flash (configurable)	Newest generally available Flash model on the project; fast and cheap
Testing	pytest (22 tests)	Guards against label leakage, schema drift and broken wiring

Layer	Choice	Why
Later	FastAPI, Streamlit or React, Cloud Run, Cloud Scheduler	Serving the dashboard and running the daily batch

6. Phase 1: loading the data into BigQuery

BigQuery is Google's managed data warehouse. Data is organised into **datasets** (like folders), each holding **tables**. We created four datasets, one per stage of processing:

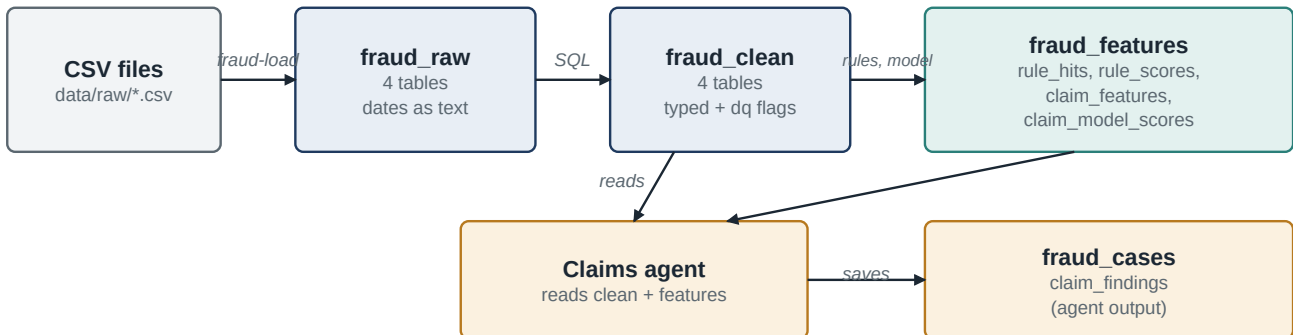


Figure 4. The four BigQuery datasets. Each stage reads the one before it and writes its own tables, so any stage can be rebuilt without touching the others.

What the loader does (uv run fraud-load)

- **Creates the four datasets** in Mumbai if they don't exist. If a dataset already exists in a different region, it stops: BigQuery cannot join tables across regions.
- **Loads each CSV into fraud_raw** with an explicit column list and types, replacing previous data so it can be re-run safely. Dates are loaded as text because the files use DD-MM-YYYY, which BigQuery cannot read automatically. A single malformed row fails the load rather than being skipped silently.
- **Builds fraud_clean** by running one SQL file per table: dates become real date columns; the word "None" in fraud_type becomes an empty value; blank policy IDs become NULL; and dq_* columns flag data-quality problems (future dates, expired "Valid" licenses, conflicting license statuses). Problem rows are flagged, never deleted.
- **Checks row counts** match across CSV, raw and clean tables (1,000 each), and fails loudly if not.

Result: all 8 tables loaded with 1,000 rows each. A check query in BigQuery reproduced the numbers from the original analysis exactly (184 / 144 / 155 / 285 fraud rows, 92 future-dated claims, 603 expired "Valid" licenses, 50 conflicting agents, 438 blank policy IDs).

7. Phase 1: the rules engine

A **rule** is a yes/no condition on a record, such as "the receipt number was already used". Each rule lives in fraud_agent/rules/rules.yaml with an ID, the table it applies to, a description, the fraud types it targets, a **severity**, and the condition written as BigQuery SQL. The engine turns every rule into SQL, runs it inside BigQuery, and writes two tables:

- fraud_features.rule_hits: one row for each time a rule fired on a record (2,679 rows).
- fraud_features.rule_scores: one row for every record in all four tables (4,000), with a score (the weight of the most severe rule hit), the highest severity, and the list of rules hit.

Severity is measured, not guessed. Each rule's severity comes from its precision on the labelled data (the share of flagged records that really are fraud): critical $\geq 95\%$, high $\geq 60\%$, medium $\geq 25\%$. Candidate rules were tested in BigQuery first; one rule (claims on a suspect policy) was dropped because its precision, 21.5%, was barely above the 18% base rate.

Guard against cheating: the engine refuses to load any rule that mentions fraud_flag, fraud_type, claim_status or approved_amount. A test checks this.

The 19 rules

Rule	Severity	Hits	Precision	Condition in plain words
POL_PREMIUM_NOT_COLLECTED	critical	45	100%	Policy issued but premium never collected
POL_BACKDATED	critical	41	100%	Start date was backdated
POL_FREE_LOOK_CANCELLED	high	49	61.2%	Cancelled inside the 15-day free-look window
POL_CANCELLED	medium	144	32.6%	Policy is cancelled
POL_AGENT_LICENSE_ISSUE	medium	330	34.2%	Sold by an agent with a non-valid license record
CLM_DOCUMENTS_ALTERED	critical	34	100%	Supporting documents altered
CLM_EXCEEDS_SUM_INSURED	critical	20	100%	Claim larger than the policy's maximum payout
CLM_EARLY_CLAIM	medium	147	27.9%	Incident within 30 days of policy start
CLM_AGENT_LICENSE_ISSUE	medium	324	34.9%	Handled by an agent with a non-valid license record
PAY_DUPLICATE_RECEIPT	critical	36	100%	Receipt number already used
PAY_NOT_REMITTED	critical	29	100%	Premium collected but not passed to the insurer
PAY_REMITTANCE_DELAY	critical	29	100%	Premium handed over more than 7 days late
PAY_BOUNCED	critical	35	100%	Payment bounced
PAY_REVERSED	critical	44	100%	Payment reversed (refund taken)
PAY_AGENT_LICENSE_ISSUE	medium	356	30.1%	Collected by an agent with a non-valid license record
GB_PREMIUM_NOT_DEPOSITED	critical	80	100%	Agent kept the premium
GB_LICENSE_NOT_VALID	high	372	72.0%	License expired, suspended, fake or never issued
GB_CUSTOMER_UNAWARE	high	273	73.6%	Customer didn't know the agent's real status
GB_REPEAT_COMPLAINTS	high	291	72.2%	Two or more complaints on the record

Baseline: how much fraud the rules catch

Recall is the share of all fraud that gets flagged. **Precision** is the share of flagged records that are really fraud. Using only certain rules gives perfect precision; adding weaker rules catches more fraud but with more false alarms.

File	Critical rules only: precision / recall	High and above	All rules: precision / recall
payments	100% / 100%	100% / 100%	36.6% / 100%
policies	100% / 55.5%	85.9% / 74.8%	33.5% / 93.5%
ghost_broking	100% / 28.1%	72.3% / 95.4%	72.3% / 95.4%
claims	100% / 29.3%	100% / 29.3%	34.9% / 82.6%

Payments and ghost broking are essentially solved by rules. Policies are mostly solved (fronting is the gap). **Claims are the weak spot:** duplicate claims, staged accidents and early-claim fraud are only caught by the weak medium rules. That is why Phase 2 started with a claims model.

8. Phase 2: the claims model

An ML model learns from labelled examples which combinations of features tend to mean fraud, and gives every claim a score between 0 and 1. We used **gradient-boosted decision trees** (many small decision trees, each correcting the previous ones), a strong standard choice for tables of data.

The features (29 per claim)

Built in BigQuery by `models/sql/claim_features.sql` into `fraud_features.claim_features`:

Group	Features
The claim	claim type, amount, days from policy start to incident, days from incident to report, claims filed by customer, witnesses, police report, documents altered
Its policy	policy type, channel, state, sum insured, premium, claim-to-sum-insured ratio, claim-to-premium ratio, backdated, premium not collected, free-look cancelled
Claim history	claims on the same policy, by the same customer, through the same agent; days since the previous claim on the policy and by the customer
Across files	agent's policy count, customer's policy count, agent's non-valid license records, agent's total complaints, critical red flags on the same agent and customer in the other three files
Excluded on purpose	<code>fraud_flag</code> , <code>fraud_type</code> , <code>claim_status</code> , <code>approved_amount</code> (outcomes) and <code>policy_status</code> (a cancellation can itself be the result of a fraud finding). A test enforces this.

How it was tested honestly

The claims were split into five groups **by agent**. The model was trained five times, each time on four groups and scored on the fifth, so every claim's score comes from a model that **never saw that agent's claims**. This mimics a new agent appearing in real life and avoids the model memorising individual agents. These “out-of-fold” scores are saved to `fraud_features.claim_model_scores`; a final model trained on everything is saved to `models/artifacts/claims_model.joblib`.

Results

PR-AUC (a 0–1 measure of how well a score ranks fraud above honest claims): **model 0.571**, rules 0.511, random guessing 0.184.

Approach	Claims flagged	Precision	Recall
Rules: critical only	54	100%	29.3%
Rules: any rule	436	34.9%	82.6%
Critical rule OR model score ≥ 0.64	79	82.3%	35.3%
Critical rule OR model score ≥ 0.42	124	61.3%	41.3%

Fraud type (at the 82% precision setting)	Recall	Why
Fake Supporting Documents	100%	The altered-documents flag
Exaggerated Claim Amount	70%	Up from 61% with rules alone; claim-to-sum-insured ratio
Early Claim Fraud	12%	Many honest claims are also filed early
Staged Accident	8%	No signal in the data
Multiple/Duplicate Claims	4%	No signal in the data

Most important features: altered documents, claim-to-sum-insured ratio, days from policy start to incident, and the agent's complaints and license problems.

What this means

The model adds a little (29% to 35% recall at over 80% precision), but no model can find fraud that leaves no trace in the data. For the hard types the plan moves to agents, which can combine evidence across files the way an investigator would. The score cut-offs were chosen on the same data they were measured on, so the precision figures are slightly optimistic.

9. Phase 3: the claims agent

An **AI agent** is a language model that can call tools in a loop: it decides what to look up, reads the result, decides what to look up next, and finally writes an answer. Ours is built with **Google ADK** (Agent Development Kit) and runs on **Gemini 3.8 Flash** through Vertex AI, billed to the project's Google Cloud credits.

Its tools (fraud_agent/tools/bigquery_tools.py)

Tool	What it returns
get_claim	The claim, its rule hits, its model score, data-quality flags, and a summary of its policy
find_related_claims	Other claims by the same customer, on the same policy, or of the same type via the same agent, within 180 days; exact-duplicate signals (same amount or date); and the baseline : how common related claims are overall (53%)
get_agent_profile	The agent's license statuses, complaints, record counts, red flags in every file, average claim score, and the baseline : 32% of all claims have an agent with a license issue
get_customer_history	All the customer's policies, claims and payments, each with rule hits
get_policy	The policy, its rule hits, and every claim and payment on it

Safety built into every tool: read-only; queries are parameterised (the agent's text is never pasted into SQL); each query is capped at 100 MB billed; and the label and outcome columns are stripped from every result, so the agent reasons only from evidence and can be scored honestly.



Figure 5. One investigation. The agent chooses which tools to call and in what order (the instructions suggest this order), then returns a findings object that is checked against a fixed schema before it is accepted.

What it returns (ClaimFindings)

Field	Content
risk_level	critical, high, medium or low
suspected_fraud_types	Zero or more of the five claim fraud types (an empty list is a valid answer)
evidence	List of facts, each citing the record ID and table it came from, quoting exact values
mitigating_factors	Facts that point away from fraud
recommended_action	reject_and_refer_to_siu (critical), hold_for_investigation (high), request_documents (medium), approve (low). SIU = Special Investigations Unit
summary	Two to four sentences an investigator can read in 20 seconds

How it has performed so far

Evaluated with `fraud-investigate --sample N`, which picks claims evenly from four groups (critical rule hit, high model score, weak rule hit only, nothing flagged) and compares the agent's answer with the hidden labels.

Run	What happened	Result
1 (20 claims)	13 failed on Vertex AI quota errors (HTTP 429)	On the 7 completed: 60% precision, 75% recall. Two honest claims rated critical because the policy or agent behind them was fraudulent
2 (20 claims)	Added automatic retries; told the agent context red flags justify “high”	“Critical”: 5 of 5 correct. “High”: only 3 of 12 were fraud. Overall 47% precision, 89% recall, fraud type correct in 7 of 9
3 (tune set, 12)	Tools now report base rates; stricter duplicate and “high” definitions	In progress at the time of writing; the 20-claim test set runs after it

Diagnosis of run 2: the agent treated any linked claim as a possible duplicate, but 51% of honest claims have one; and it treated an agent's license problem as strong, when it means fraud only about 1 time in 3. The fix gives the agent those base rates inside the tool results and tightens the definitions. To avoid tuning the prompt to the claims it is judged on, claims are now split in half by a fixed hash: prompts are tuned on the “tune” half and reported on the “test” half.

10. The full multi-agent design (target)

The claims agent is the first of several. The finished system uses a team of agents, each a specialist, coordinated by an orchestrator. In ADK terms, specialists are *LlmAgents* exposed to the orchestrator as tools (*AgentTool*), and the pipeline is a *SequentialAgent*.

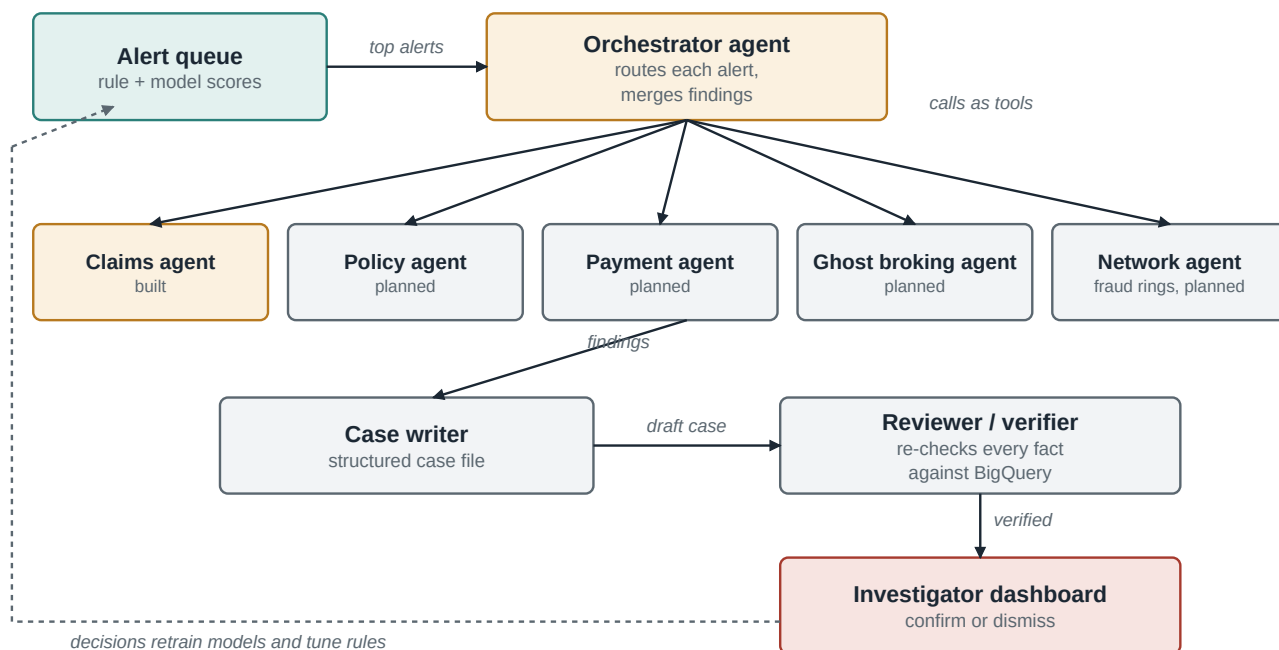


Figure 6. The target design. Each alert goes to the relevant specialists; a case writer combines their findings; a reviewer verifies every cited fact against the data (guarding against invented evidence); a person decides; decisions flow back as new labels.

Agent	Job	Status
Claims agent	Duplicate, staged, fake-document, exaggerated and early claims	Built; being calibrated
Policy agent	Fake policies, backdating, fronting, free-look abuse	Planned
Payment agent	Fake receipts, bounced cheques, refund abuse, premium diversion	Planned
Ghost broking agent	Fake or cloned agents, expired or no license, premium pocketing	Planned
Network agent	Rings: one agent or customer connected to many flagged records	Planned
Orchestrator	Decides which specialists each alert needs; merges results	Planned
Case writer	Produces the final case file for the investigator	Planned
Reviewer	Re-queries every piece of evidence; rejects anything unverifiable	Planned

11. Status, open issues and next steps

Phase	Work	Status
1	Project set-up, BigQuery datasets, loader, cleaning SQL, data-quality flags	Done
1	Rules engine (19 rules), baseline report	Done
2	Claims features and model, honest agent-grouped testing, scores in BigQuery	Done
2	Models for other files	Skipped: rules already catch 95–100%
2	Graph features (fraud rings)	Planned
3	Read-only BigQuery tools, claims agent, evaluation harness	Done
3	Calibrating the claims agent (tune/test split)	In progress
3	Other specialists, orchestrator, case writer, reviewer	Next
4	API, investigator dashboard, feedback loop, deployment (Cloud Run / Agent Engine)	Later

Open issues

- **Vertex AI quota.** The project's default Gemini quota is too low for batch runs; each claim needs several model calls. Request an increase in the Cloud console under IAM & Admin → Quotas, or accept slower runs.
- **gcloud in your terminal** uses macOS's Python 3.9, which the Google Cloud CLI no longer supports. Set `CLOUDSDK_PYTHON=/opt/homebrew/bin/python3.12`.
- **7 old rows** in `fraud_cases.claim_findings` have findings stored as double-encoded JSON (fixed in the code; the rows can be deleted with the command provided in the session).
- **Synthetic data.** Accuracy numbers will not transfer to real data; the pipeline will.

12. How to run everything

From the project folder /Users/neoxdev/Documents/FraudDetectionAiAgent:

```
uv sync                # install dependencies
uv run pytest          # 22 offline tests
uv run fraud-load      # CSVs -> fraud_raw -> fraud_clean
uv run fraud-rules     # rules -> rule_hits, rule_scores + report
uv run fraud-rules --dry-run # validate rule SQL only
uv run fraud-train-claims # claims model + report + scores
uv run fraud-investigate CLM000004 # one claim, prints findings
uv run fraud-investigate --sample 20 # evaluate on the tune split
uv run fraud-investigate --sample 20 --split test --save
uv run adk web fraud_agent/agents # chat with the agent in a local web UI
```

Project layout

```
data/raw/                the four CSV files
fraud_agent/config.py    settings from .env (project, region, model)
fraud_agent/ingest/      schemas.py, load_bigquery.py, sql/*.sql
fraud_agent/rules/       rules.yaml, engine.py
fraud_agent/models/      sql/claim_features.sql, claims_model.py, artifacts/
fraud_agent/tools/       bigquery_tools.py (read-only agent tools)
fraud_agent/agents/      schemas.py, claims_agent/agent.py, run_claims.py
tests/                   schema, rules, model and agent tests
docs/                   this document
```

Settings (.env)

```
GCP_PROJECT_ID=project-8e0caa5b-233c-4e40-823
BQ_LOCATION=asia-south1
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=project-8e0caa5b-233c-4e40-823
GOOGLE_CLOUD_LOCATION=global
AGENT_MODEL=gemini-3.8-flash
```

13. Glossary

Term	Meaning
Label	The known answer for a record (fraud_flag, fraud_type), used to train and test
Label leakage	Using information that only exists after the answer is known (e.g. claim_status); inflates results dishonestly
Base rate	How common fraud is overall (e.g. 18.4% of claims); any signal must beat it to be useful
Precision	Of the records flagged, the share that are really fraud
Recall	Of all the fraud, the share that gets flagged
PR-AUC	One number (0–1) for how well a score ranks fraud above honest records; random = the base rate
Out-of-fold score	A score from a model that did not see that record (or its agent) during training
Severity	critical / high / medium, set from a rule's measured precision
Data-quality flag (dq_*)	Marks a suspicious or impossible value; not evidence of fraud
Agent (AI)	A language model that calls tools in a loop to complete a task
Agent (insurance)	A salesperson who sells policies; identified by agent_id
Google ADK	Google's Agent Development Kit, the framework the AI agents are built with
Vertex AI	Google Cloud's AI platform; serves the Gemini models
SIU	Special Investigations Unit: the insurer's fraud investigation team
Free-look period	Legal window (15 days here) in which a new policy can be cancelled for a refund
Ghost broking	Selling insurance without a valid license or with a fake agent identity